AWS KRUG · 2026. 10. 27

문서를 이해하는 실전 AI 에이전트

aws-samples의 IDP를 서울 리전 격리 VPC에 올려 본 이야기

김재윤 · SK Inc. AX

github.com/aws-samples/sample-aws-idp-pipeline

오늘 순서

01 왜 IDP로 시작했나

02 IDP는 어떻게 생겼나

03 데모 라이브로 다섯 가지

04 서울로 옮기기

05 올리고 나서 터진 것들

06 도면이 너무 클 때

07 사내에 올리기 전에

08 정리, Q&A

도면번호, 리비전, 작성일.
아직 사람이 보고 옮겨 적습니다.

하고 싶은 것

도면과 도면목록, 사양서가 서로 맞는지 AI가 먼저 찾고 사람은 검토만

걸리는 것

한 장에 표, 기호, 치수가 빽빽하고 PDF, Excel, 스캔본이 섞여 있다

처음부터 만들까, 샘플로 시작할까

OCR만 떼어서 직접
가볍긴 한데, 검색이랑 그래프, 에이전트를 다 다시 짜야 한다

샘플을 통째로
배포하면 웹 UI와 챗봇까지 나온다. 쓸 만한지 빨리 볼 수 있다

일단 원본 그대로 올려 보고, 고치는 건 따로 하기로

aws-samples는 AWS 정식 서비스가 아니라서 고치고 유지하는 건 결국 우리 몫이다.

IDP가 하는 일

올리면

문서, 이미지, 영상, 음성, 웹 URL


S3


SQS

쪼개서 분석

페이지, 세그먼트 단위로 OCR과 LLM 분석


Step Functions


Lambda


SageMaker OCR


Bedrock BDA


Transcribe

쌓아 두고

벡터 검색과 지식 그래프에 저장


S3 LanceDB


Neptune


DynamoDB

물어본다

에이전트가 출처를 달아 답하고 리포트도 만든다


AgentCore


Claude

AWS CDK로 짠 Nx 모노레포. VPC, Storage, Workflow, MCP, Agent처럼 스택이 나뉘어 있어서 필요한 것만 골라 배포할 수 있습니다.

전체 구성 원본과 비교

추가 바꿈 원본에만 있음

서울 리전 ap-northeast-2

VPC 밖

- Cognito
- CloudFront VPC Origin
- S3 웹 호스팅, 업로드

VPC

Private 서브넷

- NLB → ALB
- ECS Fargate Backend
- Lambda 30개+
- Neptune
- ElastiCache
- Private REST API

VPC Endpoint 10여 개
밖의 서비스는 전부 여기로

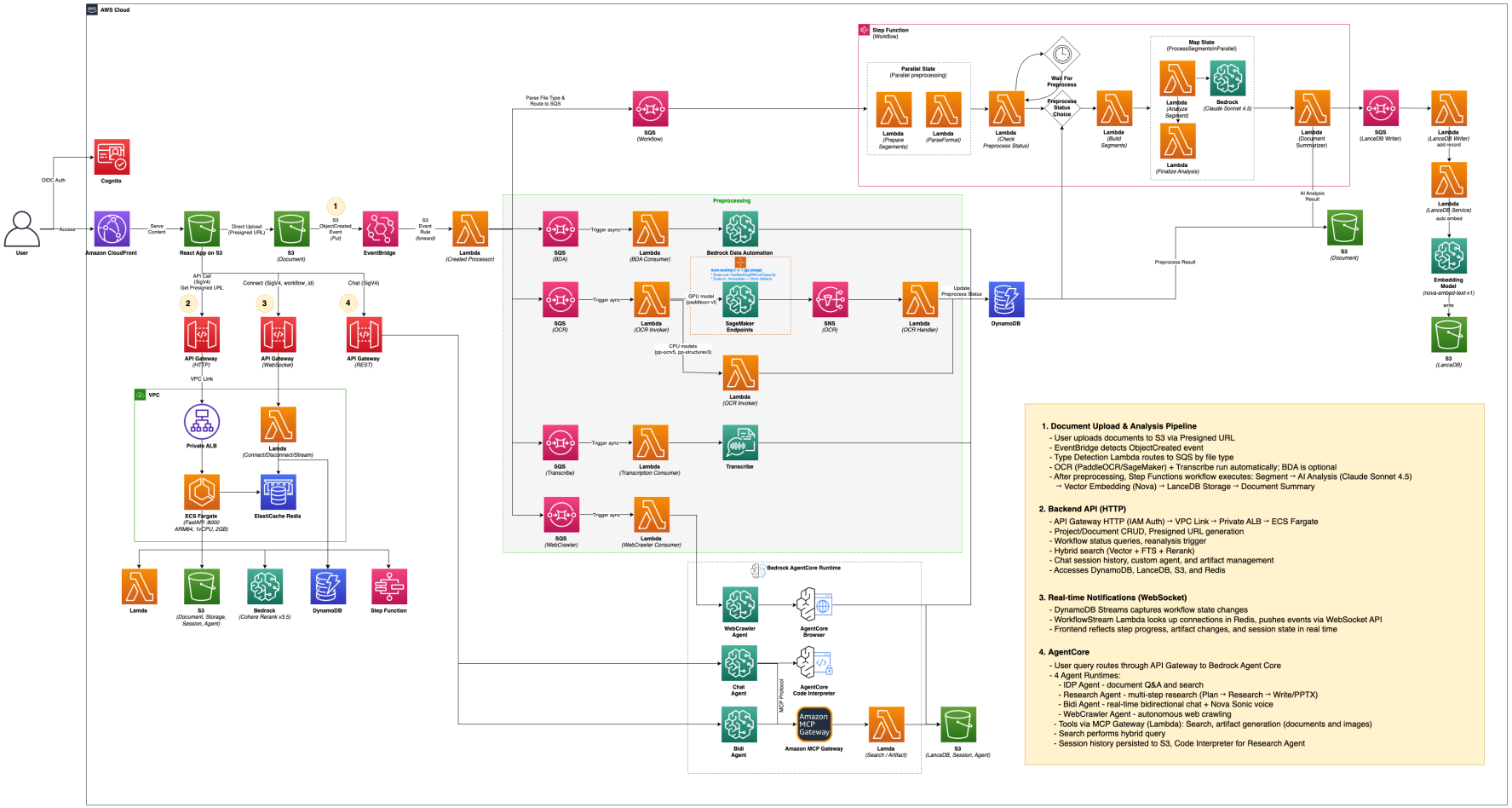
~~**NAT Gateway**~~
걷어냄, 인터넷으로 나갈 길 없음

관리형 서비스 · Endpoint로 접근

- Step Functions
- SQS
- DynamoDB
- SageMaker OCR
- Transcribe
- AgentCore
- **S3**
  Express One Zone → Standard
- **Bedrock**
  Claude는 global 추론, BDA는 묶바이

BACKUP

원본 IDP 아키텍처



출처: github.com/aws-samples/sample-aws-idp-pipeline README (2026년 10월 main 기준). 원본은 NAT Gateway가 있는 VPC 구성입니다.

문서 하나가 검색되기까지

1 

업로드

Presigned URL로 S3에 바로. 백엔드를 안 거친다

2 

트리거

S3 이벤트 → SQS → Lambda → Step Functions

3 

타입 판별

파일 형식 보고 OCR, BDA, Transcribe로 나눔

4 

동시성 제어

DynamoDB 전역 락. 세그먼트 분석은 한 번에 한 문서만

5 

세그먼트 분석

Map으로 페이지마다 병렬 호출 → Bedrock Claude

6 

문서 요약

document-summarizer → Bedrock

7 

그래프

엔티티와 관계를 뽑아 Neptune에

8 

벡터 색인

SQS → Titan 임베딩 → S3 위의 LanceDB

9 

완료

DynamoDB 상태 갱신, 화면에 알림

4번과 5번은 장애 얘기할 때 다시 나옵니다

파일마다 가는 길이 다르다

입력	처리	붙는 서비스
PDF, 이미지, 도면	OCR	PaddleOCR (Rust Lambda / SageMaker GPU)
BDA 대상 문서	구조 파싱, 필드 생성	Bedrock Data Automation
영상, 음성	전사, 장면 분석	Amazon Transcribe, TwelveLabs Pegasus
웹 URL	크롤링	WebCrawler Agent (AgentCore)
텍스트, DXF	그대로	Lambda
그 밖의 문서	format-parser	Lambda
Excel, CSV	아예 다른 파이프라인	Lambda + DuckDB, Parquet

검색은 두 군데서 한다



LanceDB on S3

Titan Embed v2로 만든 1024차원 벡터에 키워드 검색을 섞는다. 클러스터 없이 S3에 두고 Rust Lambda가 서빙한다.



Amazon Neptune

문서에서 뽑은 엔티티와 관계가 들어간다. “이 장비랑 연결된 건?” 같은 질문을 받는다.

질문 → Search MCP → LanceDB + Neptune → Haiku가 중간 정리 → 최종 답변

채팅은 AgentCore 위에서 돈다



답변

도구 결과를 모아 Claude가 스트리밍으로 답한다. 세션은 S3에 남는다.

리포트, 파일

Research Agent가 만들어 S3에 두고 Presigned URL로 내려준다.

브라우저가 AgentCore를 직접 부르는 구조라, 보안 기준이 빡빡하면 백엔드를 거치게 바꾸는 편이 낫다.

Excel은 SQL로 묻는다

올리면

1. 시트마다 검사. 병합 셀, 이미지, 차트가 있으면 거부
2. 통과한 시트는 Parquet로 바꿔 S3에
3. DuckDB로 스키마와 샘플을 훑는다
4. Claude가 설명, 컬럼 사전, 쿼리 예시를 쓴다
5. LanceDB 데이터셋 카탈로그에 색인

물으면 (data MCP)

`search_datasets`

관련 데이터셋 찾기

`describe_dataset`

스키마와 컬럼 사전 읽기

`run_sql`

Lambda 안의 DuckDB가 S3 Parquet를 바로 조회

첫 행이 헤더여야 하고, 한글 컬럼명은 영숫자로 바뀝니다. 실무 엑셀은 대부분 손을 좀 봐야 해요.

모델은 용도별로 나눠 쓴다

어디에	모델
채팅, 세그먼트 분석, 문서 요약	Claude Sonnet 5 (채팅에선 Opus 4.8도 선택 가능)
페이지 설명, 엔티티 추출, 검색 결과 정리	Claude Haiku 4.5
벡터 임베딩	Titan Text Embeddings V2
영상·PPT 스크립트 추출	Amazon Nova 2 Lite
영상 장면 분석	TwelveLabs Pegasus 1.2
실시간 음성 대화	Amazon Nova 2 Sonic
문서 구조 파싱, OCR	Bedrock Data Automation

서울 포크 기준입니다. 원본(us-east-1)은 이보다 더 많이 쓰는데, 서울에 없는 건 04에서.

직접 보겠습니다

1. 도면 PDF 올리고 분석되는 과정 보기
2. 도면에 물어보기
3. 자재목록 엑셀에 SQL로 묻기
4. 도면이랑 목록이 안 맞는 곳 찾기
5. 시간 남으면, 큰 도면이 실패하는 장면

데모 환경

개인 AWS 계정, 서울 리전(ap-northeast-2)에 배포

DEXPI 공개 예제(CC BY 4.0) 기반 가상 도면과 가상 엑셀만 사용

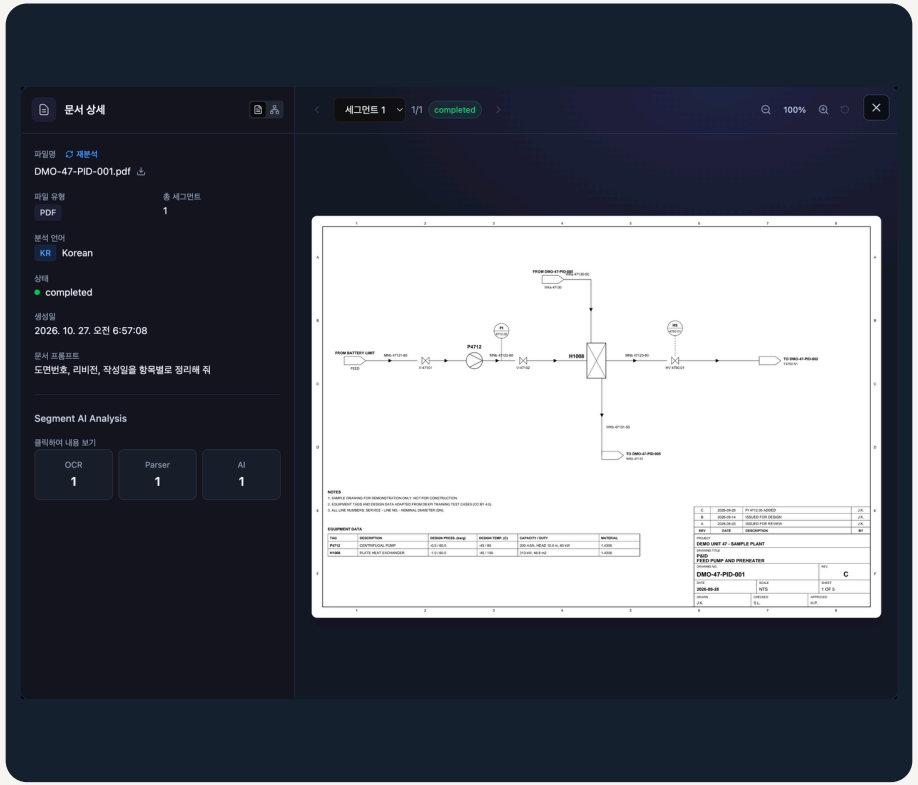
모델은 Sonnet 4.6, 임베딩은 Titan Embed V2

분석은 몇 분 걸려서 결과는 미리 돌려 둔 걸로

DEMO 1

도면 PDF를 올리면

1. 프로젝트 만들고 DEXPI 공개 예제 도면 PDF 업로드
2. 업로드됨 → 분석 중 → 완료로 바뀌는 것 보기
3. 콘솔에서 Step Functions 실행 그래프 열기
4. 다 된 문서에서 페이지별 분석 결과 보기



실제 화면 · 완료된 문서의 페이지별 분석 결과

DEMO 2

도면에 물어보기

DMO-47-PID-003 도면의 리비전이랑 작성일 알려줘

P4711 펌프 설계 용량이랑 양정을 출처 페이지랑 같이 보여줘

P4711 펌프랑 연결된 장비 전부 찾아줘

세 번째는 Neptune 그래프를 탄다

P4711 펌프 설계 용량 및 양정

항목	값
TAG	P4711
펌프 타입	변위 펌프 (DISPLACEMENT PUMP)
설계 용량 (Capacity)	420 m³/h
양정 (Head)	40.0 m
동력 (Power)	84 kW
설계 압력	-1.0 / 35.0 barg
설계 온도	-45 / 100 °C
재질	1.4306

실제 화면 · P4711 설계값과 출처 도면

DEMO 3

자재목록 엑셀에 SQL로 묻기

문서 추가 › **데이터** 탭에 가상 자재목록 엑셀 (파일 탭 아님)

설계압력 10bar 넘는 항목 몇 개야?

태그번호 중복된 행 있어?

에이전트가 만든 SQL을 펼쳐서 보여주기



실제 화면 · 10 barg 초과 27개

DEMO 4

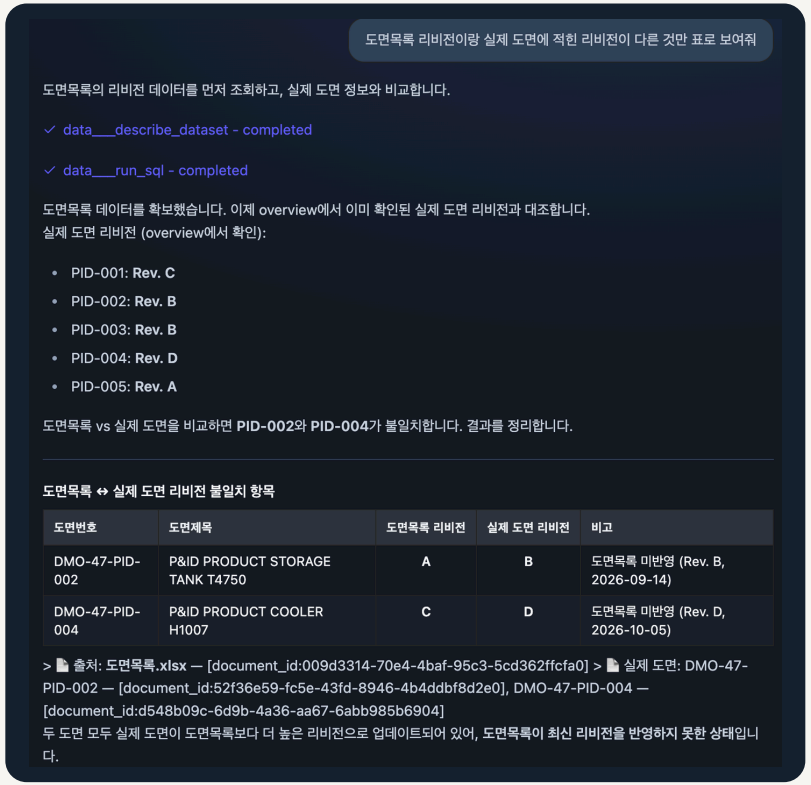
도면이랑 목록이 안 맞는 곳 찾기

준비 분석 프롬프트에 “도면번호, 리비전, 작성일을 항목별로 정리해 줘”

넣기 도면 PDF 여러 장 + 도면목록 엑셀을 한 프로젝트에

도면목록 리비전이랑 실제 도면에 적힌 리비전이 다른 것만 표로 보여줘

나오는 건 확정이 아니라 사람이 볼 후보 목록



실제 화면 · 불일치 2건: 002, 004

DEMO 5 · 로그로 보기

빽빽한 도면 한 장이 실패하는 순간

1. 표와 치수가 가득한 대형 도면 PDF 업로드
2. 그 페이지 세그먼트만 실패로 남는 것 확인
3. CloudWatch에서 segment-analyzer 로그 열기

에러로 멈추지 않고 **실패 결과만 남긴 채** 다음 단계로 넘어갑니다. 그 페이지는 검색에서 빠져요

```
/aws/lambda/idp-v2-segment-analyzer

Agent execution error

MaxTokensReachedException

max_tokens limit reached

→ success: false
```

로그 형태 예시 · 실제 프로젝트에서 본 예외

집에서 해 보실 분들께

github.com/aws-samples/sample-aws-idp-pipeline

1 처음엔 리전 바꾸지 말고 기본값으로 올리세요. 한국어 배포 가이드가 잘 돼 있습니다.

오늘 데모는 서울 리전에 맞게 고친 버전이에요. 뭘 고쳐야 하는지는 바로 다음 파트에서.

2 모델 호출이 거부되면, Marketplace 구독형은 권한 있는 사람이 콘솔에서 한 번 호출하면 풀립니다.

신규 모델은 계정팀에 요청해야 할 수도 있어요.

3 다 쓰면 스택 삭제.

Neptune, ElastiCache, SageMaker 엔드포인트, Interface Endpoint는 켜 두기만 해도 돈이 나갑니다.

서울에 없는 것들

원본이 쓰는 것	서울에서는	그래서
S3 Express One Zone	없음	S3 Standard로
Nova 2 Multimodal Embeddings	없음	Titan Embed V2로
Cohere Rerank 3.5	없음	리랭킹 단계를 뺌
Nova 2 Sonic (음성)	없음	도쿄 리전 호출
Bedrock Data Automation	소스 리전 아님	뭄바이로 우회
TwelveLabs Pegasus	us. 프로파일 불가	global. 프로파일로
AgentCore 웹 검색 커넥터	없음	기능을 끔
Claude in-region 호출	구형 모델만	global 교차 리전 추론

작업할 때 기준이라 지금은 달라졌을 수 있습니다.

원본에서 바꾼 것

13건

서울에 없는 걸 우회 6

임베딩 교체, Rerank 제거, Nova Sonic은 도쿄, Pegasus는 global, S3 Express → Standard, BDA는 묶바이

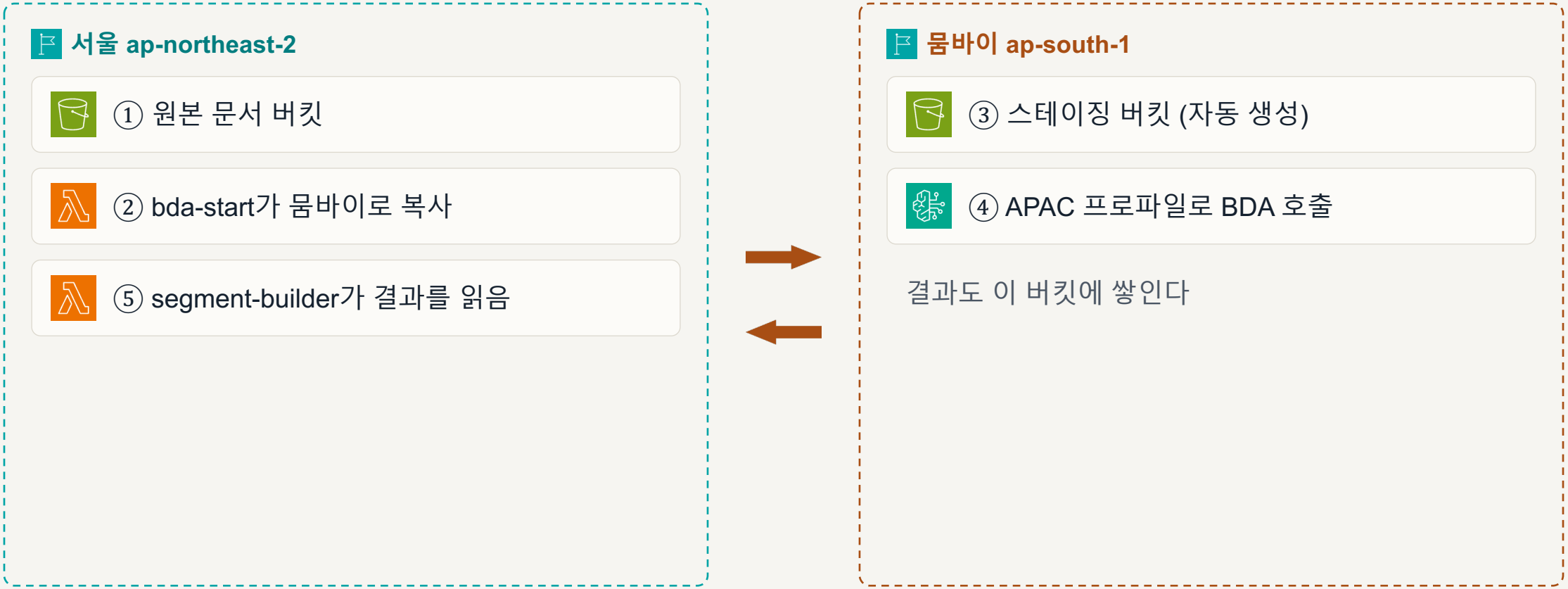
코드에 박힌 리전 버그 4

Presigned URL 엔드포인트, OCR 이미지 ECR 리전, BDA용 버킷 정책, OCR 엔드포인트 이름 주입

운영하려고 바꾼 것 3

리소스 이름 prefix, AZ 명시와 CIDR 축소, 바뀐 스택만 배포하는 CI/CD

BDA는 뭄바이를 다녀온다



BDA는 입출력 버킷이 엔드포인트와 같은 리전에 있어야 해서 이렇게 됐습니다. 대신 데이터가 잠깐 다른 리전에 머뭅니다.

코드에 박혀 있던 us-east-1

Presigned URL

리전 없이 만든 클라이언트가 us-east-1 URL을 줘서 서울 버킷에서 CORS 500

```
boto3.client("s3", region_name=region,
endpoint_url=f"https://s3.{region}.amazonaws.com",
config=Config(signature_version="s3v4"))
```

OCR 이미지 빌드

Dockerfile 베이스 이미지의 ECR 리전이 고정이라 서울에서 빌드 실패

```
FROM {dlc-account}.dkr.ecr.${region}.amazonaws.com/
pytorch-inference:2.2.0-gpu-…
```

BDA 버킷 정책

bedrock 서비스 주체를 안 열어 주면 AccessDenied

```
principals: [new ServicePrincipal(
'bedrock.amazonaws.com')]
```

어제 최신 원본을 다시 올려 보니

데모 환경 배포하면서 새로 만난 것들. 원본은 계속 바뀌고, 서울은 계속 따로 확인해야 합니다

S3 Express 가용영역	use1-az4 고정. 서울은 apne2-az3 한 곳에서만 만들어져서 리전별 매핑으로
임베딩 모델	Nova 2 Multimodal Embeddings로 바뀌었는데 us-east-1 전용. 다시 Titan Embed V2로
AgentCore 웹 검색	Gateway 웹 검색 커넥터가 서울에서 안 돼서 스택 롤백. 리전 조건으로 끔
고정 안 된 버전	최신 Rust에서 ethnum 1.5.2 빌드 실패, 에이전트는 mcp 2.x가 깔려 시작하자마자 ImportError. 버전 올리고 mcp<2로 고정 (리전과 무관)
사내망 TLS 검사	Docker 빌드 컨테이너가 사내 인증서를 몰라서 패키지 다운로드 실패. 빌드를 CodeBuild로

추론까지 서울에서만 할 건가

서울 안에서만

- 데이터도 추론도 서울
- Claude는 구형 모델만 부를 수 있다
- BDA, 영상 추출, 음성 일부는 못 쓴다
- 모델이 바뀌면 분석 품질도 바뀐다

저장은 서울, 추론은 global

- 리소스와 데이터는 서울에 둔다
- 최신 Claude와 Haiku를 쓸 수 있다
- 추론 요청은 다른 리전에서 처리될 수 있다

어느 쪽이든 조직의 데이터 리전 정책부터 확인해야 합니다.

바뀐 스택만 배포하기



스택 변경이 없으면 컴파일 전에 끝납니다. Rust 빌드 15~20분을 아낄 수 있어요.

cargo-lambda Docker 레이어와 pnpm store는 CodeBuild 로컬 캐시에 둡니다.

함정 공통 construct가 바뀌면 결국 전부 배포. CloudFront WAF 때문에 us-east-1 bootstrap도 필요합니다.

올리고 나면 만나는 문제들

이런 증상이 보이면	대개 여기가 원인
PDF 분석이 끝나지 않는다	격리 VPC
한글 이름 사용자만 대화 이력이 비어 있다	다국어 처리
WebSocket 실시간 알림이 전부 403	네트워크, Private DNS
문서를 한꺼번에 올리면 전부 멈춘다	쿼터, 동시성, 락
특정 모델 호출이 거부된다	계정 설정
넓게 물으면 채팅이 대답을 안 한다	데이터 스키마
빽빽한 도면 페이지만 분석이 실패한다	모델 출력 한도 (06에서)

PDF 분석이 끝나지 않는다

```
OCR Lambda 로그

Download failed: GET https://github.com/ …/pdfium-linux-x64.tgz
```

왜

PDF 렌더링 crate가 콜드 스타트 때마다 GitHub에서 네이티브 라이브러리를 받으려 했다. NAT가 없으니 매번 실패.

어떻게

bundled feature를 켜서 빌드할 때 실행 파일에 넣었다.

한글 이름 사용자만 대화 이력이 비어 있다

홍길동 → `í\x99\x8dê¸¸ë\x8f\x99`

왜

미들웨어가 JWT의 username을 HTTP 헤더에 다시 넣었는데, ASGI는 헤더를 latin-1로 읽는다. 저장할 땐 원래 이름, 조회할 땐 깨진 이름이라 결과가 늘 0건.

어떻게

사용자 키를 Cognito sub(불변 UUID)로 바꾸고, 헤더 대신 request.state로 넘겼다.

WebSocket 알림이 전부 403

Private REST API 때문에 execute-api Endpoint의 **Private DNS**를 켜면, VPC 안의 *.execute-api 호출이 전부 그 엔드포인트로 갑니다. 그런데 WebSocket의 PostToConnection은 REGIONAL API라 여기서 막혀요.

선택지	좋은 점	아쉬운 점
알림 Lambda만 NAT 연결	제일 빠르다	NAT 없는 원칙이 깨진다
Lambda를 VPC 밖으로	NAT 없이 유지	연결 저장소를 Valkey에서 DynamoDB로, 일이 꽤 된다
폴링으로 전환	구조가 단순해진다	상태가 몇 초 늦게 보인다

문서를 한꺼번에 올렸더니 전부 멈췄다

문서 하나에서 세그먼트 30개를 동시에 Bedrock 호출

⬇

분당 쿼터 초과, 쓰로틀링과 재시도가 쌓임

⬇

일부 Lambda가 15분 타임아웃으로 죽음

⬇

에러 이벤트에 단계 정보가 없어서 **락 해제 코드가 안 돎**

⬇

뒤에 있던 문서는 전부 대기 ↔ 확인을 무한 반복

IDP는 DynamoDB 전역 락으로 세그먼트 분석을 한 번에 한 문서씩만 돌립니다.

고친 것

- 단계 정보가 없어도 락을 푸는 fallback
- 걸려 있던 락은 DynamoDB에서 직접 지움
- Bedrock 쿼터 증설 요청
- 동시 호출 수를 낮추고 Map 실패 허용률 설정

그래서 쿼터를 다시 공부했다

이름	뜻	메모
RPM	분당 호출 횟수	토큰 크기와 상관없다
TPM	분당 입력+출력 토큰	문서 분석은 여기서 먼저 막힌다
TPD	하루 토큰 총량	대부분 못 늘린다

함정 max_tokens를 크게 잡으면, 다 안 써도 요청이 끝날 때까지 그만큼 쿼터를 먼저 잡아 둡니다. 동시 요청이 쓸 몫이 줄어들어요.

모델 접근 거부도 두 종류. Marketplace 구독형은 권한 있는 사람이 콘솔에서 한 번 호출하면 풀리고, 신규 모델은 계정팀이 열어 줘야 할 수 있습니다.

넓게 물으면 채팅이 대답을 안 한다

Search MCP Lambda

```
LanceDB Lambda error: … signal: aborted
```

⬇

LanceDB Lambda (Rust)

```
panicked at src/db/model.rs:89:60
called `Option::unwrap()` on a `None` value
```

⬇

진짜 원인

탐색이 넓어지면 entity_id 컬럼이 없는 예전 배치가 섞여 들어온다

컬럼이 없으면 패닉 대신 빈 결과를 돌려주도록 세 함수를 고쳤습니다. 타임아웃인 줄 알았는데, 프로세스가 죽고 있었어요.

도면 한 장이 출력 한도를 넘는다

수만 자 → **16,384**

대형 도면 한 장의 OCR 텍스트

출력 한도 (max_tokens)

프롬프트는 “Extract EVERY text element, table …”. 모델은 수백 행짜리 표를 그대로 다시 쓰다가 끊기고, 원본 코드는 그 페이지를 실패로 처리합니다. 검색에서 통째로 빠져요.

뭘 포기할지 골라야 했다

방법	얻는 것	잃는 것
잘린 응답이라도 저장	구조는 그대로	뒷부분이 빠진다
요약하라고 시키기	한도 안에 들어온다	세부값으로 검색이 안 된다
도구 호출 횟수 제한	한도 안에서 끝난다	일부 영역이 빠진다
원문을 따로 임베딩	검색 누락이 없다	파이프라인을 바꿔야 한다

그래도 LLM을 거치는 이유는 있습니다. OCR 오탈자를 고치고, 섹션별로 구조를 잡고, 이미지를 직접 보고 OCR이 놓친 걸 채워 주니까요.

실패 대신 원문으로 색인한다

MaxTokens 초과

→

커스텀 프롬프트 없음
OCR·파서 원문을 분석 결과로 저장하고 성공 처리

커스텀 프롬프트 있음
커스텀 지시만 가볍게 따로 재시도. 원문도 같이 저장

→

임베딩 전에 7,500자 청킹
줄바꿈 기준으로 자르고 청크마다 임베딩

덤으로 정상 분석 결과도 7,500자가 넘으면 뒷부분이 검색 안 되던 문제가 같이 풀렸습니다.

아직 모르는 것 수만 자짜리 원문이 그래프 품질에 주는 영향. 실제 문서로 더 봐야 합니다.

Excel도 같은 벽에 부딪힌다

PDF는 페이지 하나가 세그먼트 하나인데, Excel은 **시트 하나가 세그먼트 하나**예요. 행이 많으면 바로 한도를 넘고, 시트 안 이미지는 무시됩니다.

시트를 나눠서 처리

- 이미지가 있는 시트는 도면처럼 이미지 분석
- 정형 표는 행 단위 청킹이나 Text2SQL
- 병합 셀 양식은 기존 방식 그대로

아직 못 푼 것

- 빨간 셀, 취소선 같은 서식이 가진 의미
- 병합 셀 양식은 구조가 무너진다
- 이미지와 행을 정확히 짝짓기

사내에 올리기 전에 확인할 것

- [ ] **네트워크** NAT 없이 막을 거라면 런타임 다운로드와 Private DNS 충돌부터
- [ ] **데이터 리전** BDA 우회와 교차 리전 추론으로 데이터가 어디를 거치는지
- [ ] **인증** 사내 IdP로 바꾸면 사용자 키와 Presigned URL 경로도 같이 바뀐다
- [ ] **퍼블릭 구간** S3·CloudFront 대신 내부 ALB 뒤 컨테이너로 옮길 수 있다
- [ ] **쿼터** 대량 처리 전에 TPM 증설, 동시 호출 수와 실패 허용률 조정
- [ ] **업스트림** 포크에서 바꾼 걸 적어 둬야 업데이트를 따라갈 수 있다

정리하면

1. **샘플은 출발점일 뿐이다.** 고치고, 유지하고, 업스트림 따라가는 건 우리 몫.
2. **리전 이식은 문서보다 코드에서 오래 걸린다.** 숨은 us-east-1을 찾자.
3. **격리 VPC는 숨은 의존성을 드러낸다.** 런타임 다운로드, Private DNS.
4. **쿼터와 동시성은 설계의 일부다.** 작은 버그 하나와 겹치면 전부 멈춘다.
5. **LLM 출력 한도는 못 피한다.** 뭘 잃을지 정하고, 조용히 실패하지 않게.

Q&A

감사합니다

김재윤 · SK Inc. AX

저장소와 한국어 문서

github.com/aws-samples/sample-aws-idp-pipeline

aws-samples.github.io/sample-aws-idp-pipeline/ko/